INDEPENDENT APPLIED MACHINE LEARNING PROJECT

Sensor Event Detection Under Channel Dropout

Evaluating when to issue or defer an automated label from two synchronized signals

James Jean | November 2026

Abstract. When one of two synchronized monitoring channels drops out, an equipment team can issue an automated event label or defer the window for review. I built a deterministic CPU benchmark to compare those choices under known synthetic labels. A training prevalence baseline, a regularized logistic regression, and a 1,357-parameter one-dimensional convolutional neural network (CNN) were compared on 1,800 later windows from 30 held-out device instances. Clean, brief, and sustained dropout views were paired by source window. Calibration and confidence thresholds were selected on disjoint devices before generating the final test windows. Under sustained dropout the CNN reached 0.517 average precision versus 0.352 for logistic regression (paired difference 0.166; 95% device-bootstrap interval 0.116–0.205). The fixed CNN policy labeled 1,438 of 1,800 windows. It made 274 errors among the issued labels (19.1%), compared with 431 errors (23.9%) if every window received a label. Deferral removed 157 would-be mistakes from automated output while also withholding 205 would-be correct labels. A descriptive cost analysis holds the rule fixed and shows how cost and review capacity can change the decision. These results support a shadow evaluation of the method.

READER MAP

Operational decision and evaluation scope 2
Data generator, splits, and model fitting 3
Policy, results, and failure analysis 5
Cost sensitivity, validation plan, and provenance 9

Evidence boundary. All signals and labels were generated by a controlled simulator. The study has no physical deployment, client data, human review outcomes, or measured financial return.

1 Operational decision

A monitoring workflow can reduce a stream to short windows and ask whether each contains an event. Two synchronized channels can agree on a pattern that looks ambiguous in either channel alone. During an outage, the surviving channel still contains information, but confidence can shift in both directions: an event can look like a normal window, and a single-channel transient can look like an event. A team must decide whether to write the automated label into a downstream workflow or send the window to a review queue. Those actions place different demands on response time, reviewer capacity, and error tolerance.

The project isolates the decision with a binary event label. Each window has known ground truth in the generator, two aligned sequences, and an explicit observed-sample mask. The same window appears as a complete observation and under two durations of loss. This lets us compare models without attributing performance differences to a changed mix of events. The nominal 32-sample-per-second rate and sample indices are simulation settings. They do not describe installed devices or tell a live team how many items arrive in an hour.

Decision compared. The issue-all option predicts event or no event at a fixed probability threshold. The selective option uses the same class rule but withholds a label when its confidence is below a validation-selected cutoff. The study treats deferral as a queue entry and checks what would have been issued in its place. The analysis does not assume a reviewer would recognize or resolve every deferred event. Both policies can miss events.

Questions. Does the sequence model rank events more accurately than a training prior and a fixed-feature baseline on later, unseen devices? How much performance is lost when one channel goes missing? At the frozen operating point, how many incorrect automated labels are withheld, how many correct labels are also withheld, and what review load follows? Ranking, probability quality, and operating results answer different parts of that decision.

Method context. The risk–coverage view follows work on selective classification [1, 2]. This benchmark applies its reject rule after training and calibration; there is no jointly trained reject head. Post-training probability calibration provides a useful framework [3]. The implementation fits an affine sigmoid (slope and intercept), rather than temperature scaling alone. The controlled experiment tests a bounded engineering choice and makes no claim of state-of-the-art performance.

2 Controlled benchmark and partitions

Generator. For each fixed seed (101, 203, 307), the generator creates 36 device instances with distinct offsets, gains, periodic nuisance, and Gaussian noise. Each device contributes 60 disjoint 64-sample windows. A Bernoulli draw with probability 0.26 assigns each planted event. Events use a signed biphasic pulse with a small inter-channel delay. Non-events may have a single peak, a same-polarity echo, or an event-like pattern in just one channel. The offsets include mild acquisition-block drift. Labels come from the generator, with no external annotation or field measurements.

Chronology and device isolation. Device assignment occurs before generating or windowing signals. A device contributes to exactly one of four time-ordered partitions. The next block starts 65 samples after the preceding last sample, leaving 64 intervening samples. Checks cover strict chronology at the partition boundary, source IDs, device disjointness, and duplicate clean signal content. The final comparison intentionally combines unseen devices with later acquisition blocks.

Table 1: Partition roles, assigned before windows are generated. Each device contributes 60 source windows.

Block	Devices / seed	Windows / seed	Across seeds	Purpose
Train	18	1,080	3,240	Scaling and model weights
Calibration	4	240	720	Affine probability mapping
Policy	4	240	720	Threshold and model choice
Final test	10	600	1,800	Evaluation only
Total	36	2,160	6,480	108 device instances

Missingness. The clean view retains both channels. Brief dropout removes 12 consecutive samples from one channel, starting at an index drawn from 16 through 36 inclusive. Sustained dropout removes the same channel at indices 6 through 57 inclusive (52 samples). The masks are nested. The channel and brief position come from a stable source ID hash and do not use the label. Loss covers 12/128 (9.4%) or 52/128 (40.6%) of the two-channel grid. This models two severities of one fault. Test labels and IDs remain identical across views and models, producing 16,200 score rows for 1,800 distinct windows.

3 Model fitting and probability calibration

Common observations. The two learned models see the same three training views and replicated labels. Channel means and standard deviations come from clean training windows. The fixed constants scale the observed values; missing samples become zero in normalized coordinates. The mask remains available so a missing value does not silently become an observation of the training mean. Predictors see observed values and masks, without source IDs, device IDs, acquisition block identifiers, or generator parameters.

Training prevalence. One event probability per seed is estimated from its training labels and remains fixed across views. Pooling three seed-specific prior estimates yields average precision near, but not necessarily equal to, pooled test prevalence.

Regularized logistic regression. Fixed features include per-channel observed fraction, mean, root mean square, standard deviation, absolute peak, adjacent roughness, lagged products, and energy and support in four contiguous segments. The feature vector also uses simultaneous support and synchronous products across channels. The pipeline standardizes the features and fits L2-regularized logistic regression ($C = 1$, up to 500 iterations) [5].

Sequence CNN. Four input planes hold two scaled signals and two masks, each of length 64. A padded convolution maps $4 \to 12$ channels with kernel width 7; a second maps $12 \to 16$ with width 5. Each is followed by ReLU. Mean and max pooling across sample positions produce 32 values. A linear readout produces one logit [4]. The model has $348 + 976 + 33 = 1,357$ parameters. It uses AdamW, binary cross-entropy on logits, batch size 128, learning rate 0.003, weight decay 0.0001, and 14 fixed epochs. One CPU thread and deterministic PyTorch algorithms are enabled. The experiment compares complete modeling pipelines; it does not isolate the effects of the architecture from the different input representations.

Calibration. The training prior bypasses calibration. Within each seed, the learned model's logits on all three calibration views are pooled and fitted by a one-variable logistic regression ($C = 10$). Given slope a, intercept b, and logit z, the calibrated estimate is $\hat{p} = (1+\exp[-(az+b)])^{-1}$. No policy or test labels enter that fit. The mapping can improve probabilistic forecasts without assuring calibration for each fault or device.

4 Frozen policy and evaluation

Let y_i be the generated event indicator, $\hat{y}_i = 1[\hat{p}_i \geq 0.5]$ the predicted label, and $s_i = \max(\hat{p}_i, 1 - \hat{p}_i)$ the confidence. A policy with cutoff t issues a label when $s_i \geq t$. Let I_i indicate an issued label and $e_i = 1[\hat{y}_i \neq y_i]$ an incorrect label. On N source windows, coverage is $\sum I_i/N$, and selective risk is $\sum I_i e_i / \sum I_i$ when any labels are issued. The decision loss is

$$L_c(t) = \frac{\sum_i I_i e_i + c \sum_i (1 - I_i)}{N}.$$

One wrong automated label costs one illustrative unit; a deferral costs $c = 0.18$ units in the frozen protocol. Equal error weights simplify the comparison and are not a claim about the relative cost of real false negatives and false positives.

Selection. The policy partition evaluates cutoffs from 0.50 to 0.95 by 0.05 for each model and seed. Each cutoff must retain at least 70% in every policy scenario. Selection minimizes equally weighted scenario loss, preferring greater coverage and then a lower cutoff in ties. Mean policy loss across seeds selects the model, with ties favoring the simpler model. Those losses were 0.140 for the CNN, 0.215 for logistic regression, and 0.268 for the training prior. The selected CNN cutoffs were 0.70, 0.65, and 0.70 by seed. The runner wrote the decisions before generating the final test windows.

Metrics. Average precision summarizes the precision–recall curve as a recall-weighted sum of precisions, grouping tied scores. Brier score is the mean squared error of the probability forecasts. Reliability plots compare mean forecasts and event fractions in ten equal-width bins, omitting empty bins and representing support. Brier reflects more than calibration alone [5]. The threshold-based metrics report coverage, error among issued labels, and decision cost.

Uncertainty. The original benchmark drew 400 paired cluster-bootstrap resamples, sampling ten devices with replacement within each seed and retaining their windows across views and models. All draws contained both classes. Percentile intervals condition on these fits, the generator, and the observed clusters. They do not include retraining uncertainty, multiplicity correction, or uncertainty on physical devices. The independent CSV verifier recomputes the paired point contrasts. The interval bounds come from the original code and were reproduced in the fresh CPU run.

5 Comparative results

The final test has 482 event windows (26.8%). The CNN has higher average precision and lower Brier score than both baselines in each condition. Its average precision falls from 0.798 to 0.517 from clean to sustained dropout. The paired CNN-minus-logistic difference is 0.166 under sustained loss (95% percentile interval 0.116–0.205), smaller than the clean difference of 0.245 (0.214–0.276). The CNN's own sustained-minus-clean difference is −0.281 (−0.310 to −0.252). A higher ranking score than the baseline does not make the detector insensitive to a channel outage.



Figure 1: Held-out ranking and probability scores, with the paired CNN-minus-logistic average precision contrast at right. Contrast bars are frozen 95% within-seed device-bootstrap percentile intervals. Each condition uses the same 1,800 windows.

Table 2: Pooled held-out average precision and Brier score (AP / Brier).

Model	Clean	Brief dropout	Sustained dropout
Training prevalence	0.265 / 0.196	0.265 / 0.196	0.265 / 0.196
Logistic regression	0.552 / 0.176	0.490 / 0.182	0.352 / 0.201
1D CNN	0.798 / 0.111	0.686 / 0.142	0.517 / 0.170

The CNN's sustained-dropout Brier score moves from 0.173 before calibration to 0.170 after calibration. The reliability view on the next page shows where forecasts and frequencies still differ. Sparse bins should not be read as precise estimates.

6 How the frozen policy behaves

Under sustained loss, the CNN issues 1,438 labels and makes 274 errors among them (19.1%). Issuing all 1,800 labels gives 431 errors (23.9%). The two rates use different denominators. Against the common total of 1,800, wrong *automated* labels fall from 431 to 274; the workflow's ultimate outcome on deferred items is unknown. The same rule issues 87.8% of clean and 87.0% of brief-dropout windows, with 11.1% and 15.1% error among those labels, respectively.

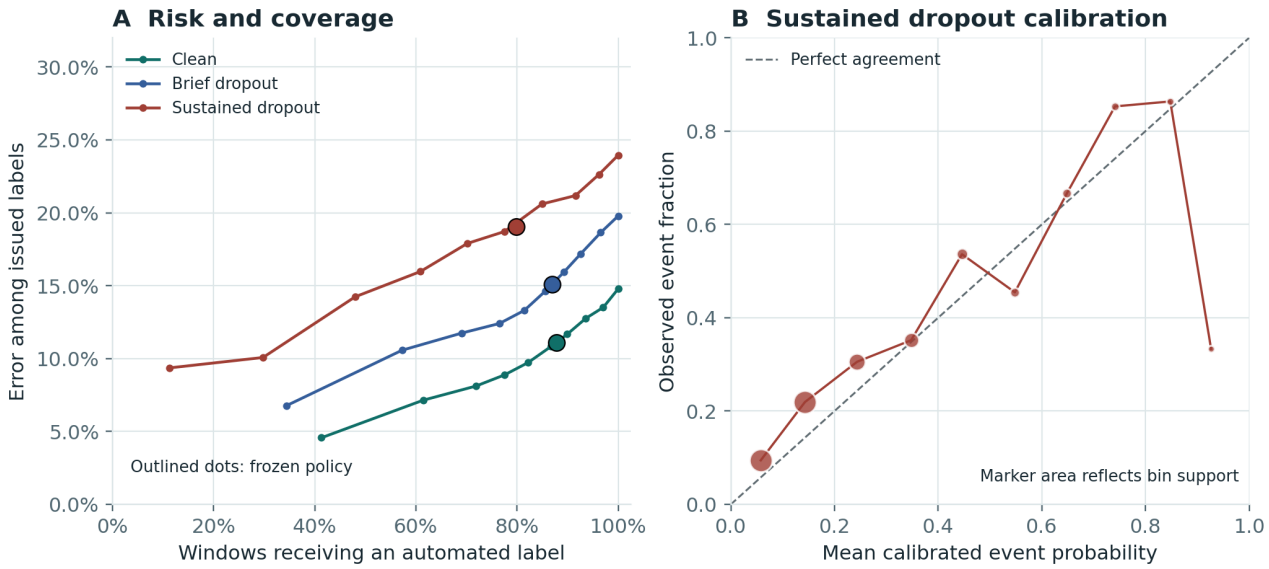


Figure 2: Left: descriptive curves use common confidence cutoffs on the saved test scores. Outlined dots show the actual rule with seed-specific cutoffs; those dots need not lie on a common-cutoff curve. Right: sustained-dropout reliability, with marker area reflecting bin support.

Missed events remain a key concern. Of the 274 issued errors under sustained loss, 260 are false negatives and 14 false positives. Issuing the deferred latent labels would add 111 false negatives and 46 false positives. Equal-cost performance can improve while the number of missed events remains unacceptable. A team should evaluate severity and set a missed-event limit before changing live decisions.

7 Failure examples and negative control

On 209 windows the CNN's prediction is correct when clean and wrong under sustained loss. The examples below are two deterministically selected positive cases, one deferred and one retained, with identical generated truth across views. They are illustrative cases rather than a representative sample. Gaps show missing observations; the unaffected channel remains visible. Scores are the saved calibrated estimates.

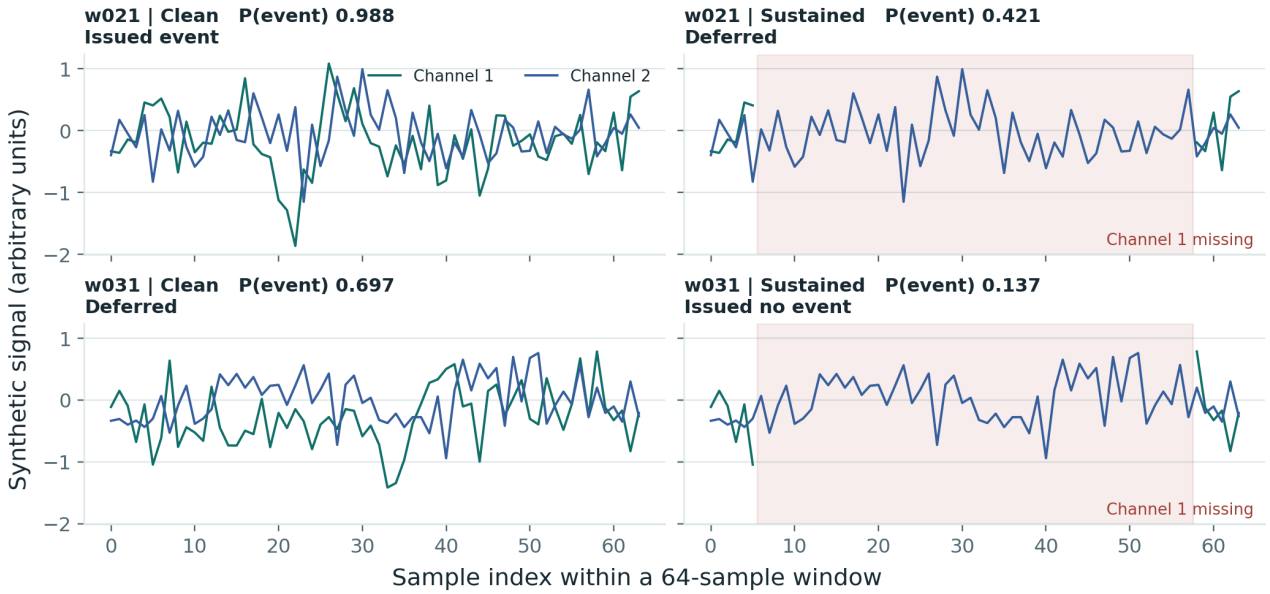


Figure 3: Positive windows from synthetic device `s101-d33`. Under dropout, `w021` shifts from 0.988 to 0.421 and is deferred. Window `w031` shifts from 0.697 to 0.137 and receives an incorrect no-event label. Signal units are arbitrary.

Confidence does not catch every mistake. A false negative survives because losing a channel can make the remaining evidence look convincingly negative. Track this failure by outage type, severity, and device, and compare it against a rule that explicitly escalates outages.

Training-label shuffle. One additional fit shuffled training labels before replicating the views and evaluated raw scores on 600 clean windows from seed 101. Average precision was 0.3643 versus 0.2717 test prevalence. That is below the learned model but above the prevalence reference. A single shuffled fit neither defines a null distribution nor rules out leakage. Separate device, time, content, and fit-source checks address specific leakage paths.

8 Cost and review capacity

This is a **post-hoc description of the unchanged policy** from released predictions. It adds no new training, threshold choice, or review outcome. Under sustained loss, the rule defers 362 windows: 157 would have had a wrong label and 205 a correct one under issue-all. That is 201 deferred items per 1,000 windows if this benchmark condition repeated; clean and brief conditions would produce about 122 and 130, respectively. The normalized counts do not forecast arrivals or staffing per hour.

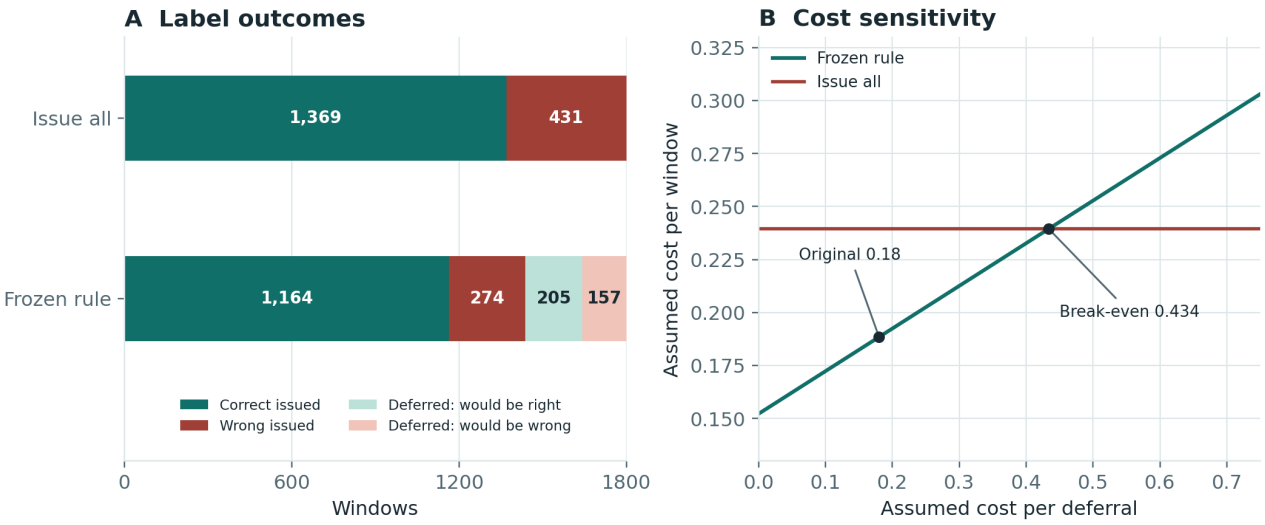


Figure 4: Outcomes on the same 1,800 sustained-dropout windows and cost sensitivity for both fixed options. Pale segments show deferred items grouped by whether their unissued label would have been right. Costs use assumed units; human review success is unknown.

Issue-all costs 431/1800 units per window under the illustrative loss. The frozen rule costs $(274 + 362c)/1800$. They meet at $c = (431 - 274)/362 \approx 0.434$. At $c = 0.18$, cost is 0.188 versus 0.239 units per window. Above break-even this simplified loss favors issue-all; it does not establish that issuing every label is safe. In practice, review time, delay, reviewer mistakes, and queue saturation may need separate costs or a different decision model.

9 Recommendation and limits

Advance to shadow evaluation. On the synthetic benchmark, retain the CNN and frozen policy as the leading candidate. For a live team, run it alongside the current workflow without changing alerts or labels. Log raw scores, masks, dropout tags, and decisions. Compare against issue-all and an explicit outage escalation rule. Review a stratified sample of both issued and deferred windows using adjudicated labels. Sampling only the queue would hide confidently wrong issued labels.

Set go/no-go criteria first. Use observed error severity and review capacity to set a missed-event ceiling with an uncertainty bound, queue volume and aging limits, and escalation for prolonged or multi-channel failures. Test on later blocks and physical devices held out by identity. Do not advance if safety or capacity limits are exceeded, uncertainty is too wide, or fault patterns differ materially. If a different cutoff is needed, choose it on a new validation partition, then test on new held-out data.

Limits. The controlled task uses two channels, one planted event family, known masks, a fixed training mixture, and one simulated acquisition regime. Real loss can be informative, affect both channels, or occur alongside clock drift. Real reviews may be slow or wrong. No comparison here establishes external accuracy, deployment readiness, improved field safety, or savings. The benchmark demonstrates a repeatable way to compare the decisions under known labels.

10 Reproduction and provenance

Frozen inputs and evidence live in `benchmark/` and `evidence/`; new analysis and figure sources live in `analysis/`. Ten base tests passed. Independent code recomputed ranking and probability metrics, coverage, and paired point contrasts from the saved scores. Two CPU processes agreed byte for byte on eleven core files and nine dependency audit reports. The project fit three seeded CNNs plus the separate shuffle control. The publication analysis adds no fit.

The public README gives exact commands, environment pins, and the dependency hash. The new analysis reads the saved score CSV, result JSON, and validation decision JSON. Its machine-readable provenance lists their SHA-256 digests. Anyone can verify the saved scores without training and compile the manuscript without running any model.

A Implementation and verification checklist

Stage	Evidence available in the publication tree
Design	Frozen configuration, source digests, seeds, loss weights and threshold grid.
Partitions	Device and time records, partition fingerprints, split checks.
Pretest decision	Validation costs, cutoffs by model and seed, selection fingerprints.
Final scoring	Full paired CSV including raw and calibrated scores, labels, issue flags and missingness.
Point estimates	Recomputable metrics, uncertainty provenance, failure examples.
Decision analysis	Counts, error-type breakdown, fixed-rule crossover and normalized review load.
Reproduction	Version pins, frozen runner, tests, exact licensed dependency, and commands.

Terminology. A device is a simulated instance within a seed. A view is a clean window or its masked copy. A deferral is the absence of an automated label at the selected cutoff. A wrong deferred label is the model's unissued thresholded prediction, not a reviewer decision.

References

[1] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017. arXiv:1705.08500.

[2] Yonatan Geifman and Ran El-Yaniv. SelectiveNet: A deep neural network with an integrated reject option. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97, pages 2151–2159, 2019. PMLR 97.

[3] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 1321–1330, 2017. PMLR 70.

[4] PyTorch contributors. PyTorch 2.7 reference: Conv1d, 2025. API documentation.

[5] scikit-learn contributors. scikit-learn 1.7 user guide: Metrics, scoring, and calibration, 2025. Metrics and calibration.